



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2024/2025

Fase 2 - Transformações Geométricas

Grupo 5

Marco Soares Gonçalves (a104614) André Filipe Soares Pereira (a104275)
Salvador Duarte Magalhães Barreto(a104520) Leonardo Gomes Alves(a104093)

30 de Março de 2025

CG

Fase 2 - Transformações Geométricas

Grupo 5

Marco Soares Gonçalves (a104614) André Filipe Soares Pereira (a104275)

Salvador Duarte Magalhães Barreto(a104520) Leonardo Gomes Alves(a104093)

30 de Março de 2025

Índice

1	Introdução	1
2	Atualização do Parser	2
3	Sistema solar	3
3.1	Anel de Saturno/Urano	3
3.2	Desenvolvimento do ficheiro XML	5
3.3	Execução da <i>demo scene</i>	6
4	Conclusão	8

Lista de Figuras

2.1	Estruturas das transformações	2
3.1	Anel composto por duas circunferências	3
3.2	Anel composto por quatro slices	4
3.3	Posicionamento da Terra e da Lua	5
3.4	Posicionamento de Saturno e o seu anel	6
3.5	Sistema solar	7

1 Introdução

Este relatório documenta a segunda fase do desenvolvimento do projeto da disciplina de Computação Gráfica. O objetivo desta fase é adicionar ao sistema da primeira fase, a funcionalidades de *rotation*, *translate* e *scale*, adicionando assim novos campos aos ficheiros XML. Para além destas novas funcionalidades, é ainda requerido a elaboração de uma *demo scene* correspondente ao sistema solar.

Ao longo desta fase, verificámos que os triângulos que compõe algumas das faces do *box* estavam invertidos, isto é, a diagonal que compõe cada quadrado estava inclinada para o lado errado. Estes erros apenas foram percetidos nesta fase, uma vez que as demo scenes desta fase esclareceram essas mesmas faces. É de notar que este "erro" apenas influencia a forma como os triângulos estavam a ser posicionadas e, na fase anterior a *box* estava a ser corretamente gerada.

2 Atualização do Parser

Com o objetivo de suportar as novas funcionalidades de transformações (*rotate*, *scale* e *translate*), tivemos que realizar algumas alterações na estrutura responsável por guardar os dados dos ficheiros XML, a estrutura **World**. Nesta adicionámos a estrutura **Transform** dentro da estrutura dos **groups**, desta forma, cada grupo do ficheiro XML pode ter a si atribuído no máximo três transformações: 1 de *scale*, 1 de *rotate* e 1 de *translate*, e pode herdar transformações caso seja um grupo filho.

```
You, last week | 1 author (You)
struct Translate {
    float x = 0, y = 0, z = 0;
    int order = -1;
};

You, last week | 1 author (You)
struct Rotate {
    float angle = 0, x = 0, y = 0, z = 0;
    int order = -1;
};

You, last week | 1 author (You)
struct Scale {
    float x = 0, y = 0, z = 0;
    int order = -1;
};

You, 2 weeks ago | 1 author (You)
struct Transform {
    Translate translate;
    Rotate rotate;
    Scale scale;
};
```

Figura 2.1: Estruturas das transformações

É importante realçar que a ordem das transformações é relevante e, desta forma, utilizá-mos um contador que associa a cada transformação dum grupo **transform** o seu índice ([1,3]). Desta forma garantimos que as transformações são executadas conforme a sua ordem no ficheiro XML.

Uma vez que é necessário que grupos filho herdem as transformações do(s) grupo(s) pai, alterámos a função responsável por guardar as figuras em memória (**parseShapes**). Esta função para além de ler os ficheiros *.3d*, é capaz agora de a cada figura que lê, associa a essa as transformações necessárias num vetor de estruturas **Transform**, para ser posicionada nas coordenadas corretas. Desta forma, quando as figuras forem desenhadas, antes dos seus pontos serem lidos, serão executadas as suas transformações.

3 Sistema solar

3.1 Anel de Saturno/Urano

Uma vez que os planetas de Saturno e Urano possuem anéis, decidimos então desenvolver o seu desenho.

Para tal, conseguimos verificar que um anel pode ser "composto" por duas circunferências, onde a circunferência com raio denominado por **inner_radius**, corresponde à circunferência interior do anel, e a circunferência com **diâmetro** denominado por **out_radius**, corresponde à circunferência exterior do anel, onde o raio efetivo dessa circunferência será a $\text{inner_radius} + \text{out_radius}$. Desta forma, o espaçamento que existe entre as circunferências definem corretamente um anel.

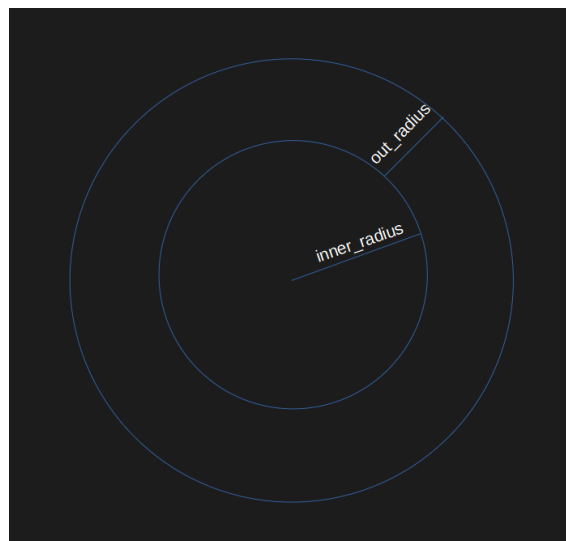


Figura 3.1: Anel composto por duas circunferências

De seguida, uma vez que o OpenGL requer que utilizemos triângulos para a construção de qualquer figura geométrica, de forma similar à nossa construção do cone e da esfera, iremos dividir a circunferência em **slices**. Imaginando então um anel composto por 4 slices, seria representado da seguinte forma.

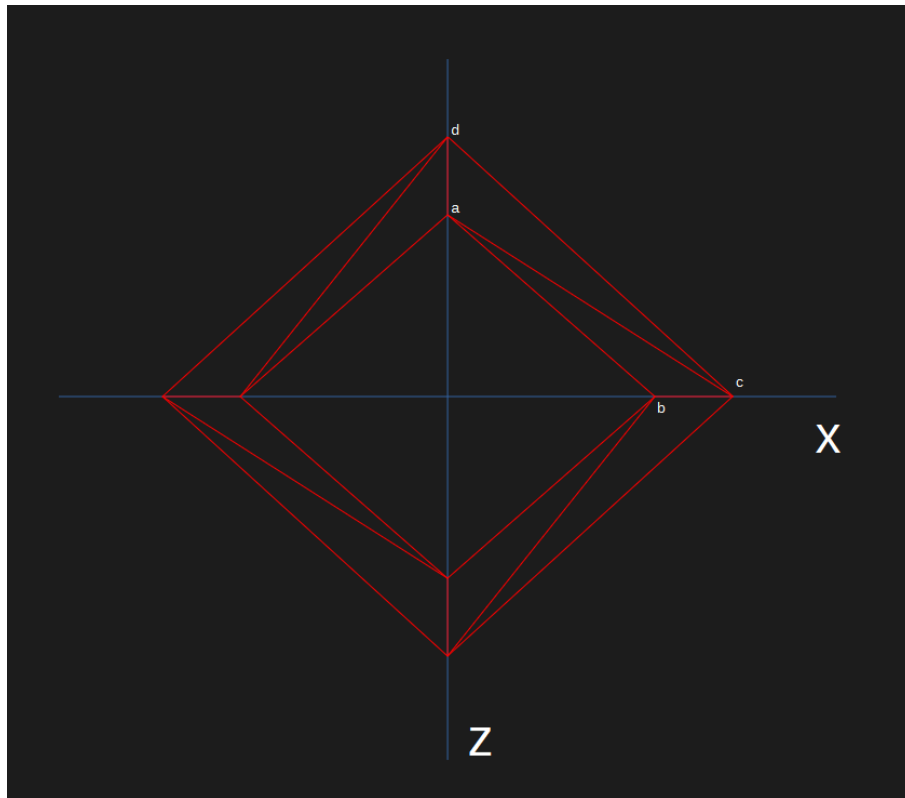


Figura 3.2: Anel composto por quatro slices

Assim, com o auxílio do desenho, e uma vez que a esfera será desenhada no plano XZ ($y = 0$), podemos concluir o seguinte acerca das coordenadas dos pontos **a**, **b**, **c** e **d** desenhados na imagem acima, que definem uma *slice* do anel:

- Ponto **a** - $X = 0$, $Z = \text{inner_radius}$
- Ponto **b** - $X = \text{inner_radius}$, $Z = 0$
- Ponto **c** - $X = \text{out_radius}$, $Z = 0$
- Ponto **d** - $X = 0$, $Z = \text{out_radius}$

Desta forma, conseguimos verificar que estes 4 pontos podem ser definidos à custa dum ângulo α , que irá variar entre $[0, 2\pi]$.

Assim, sabemos que esse ângulo começará com valor 0 e, conforme for desenhando as slices, será adicionado um valor $\alpha_{step} = \frac{2\pi}{\text{slices}}$ de forma a avançar para as coordenadas dos próximos pontos a serem desenhados. No caso da imagem acima, o valor de α nos pontos **b** e **c** é 0, e o valor nos pontos **a** e **d** será $0 + \alpha_{step}$.

Por fim, as coordenadas X e Z de cada ponto pertencente ao anel serão definidas através da seguinte fórmula, onde r será **inner_radius** ou **inner_radius + out_radius** dependendo do ponto que queirámos desenhar:

- $X = r \times \cos \alpha$
- $Z = r \times \sin \alpha$

3.2 Desenvolvimento do ficheiro XML

Com o objetivo de desenvolver a *demo scene* referente ao sistema solar, decidimos utilizar como objeto base para os planetas uma esfera de raio 1, com 10 *slices* e 10 *stacks*. Para os planetas de Urano e Saturno, decidimos utilizar dois anéis: com 15 *slices*, 1.3 de *inner_radius*, 0.5 de *out_radius* o anel de Urano; com 15 *slices*, 1.5 de *inner_radius*, 0.2 de *out_radius* o anel de Saturno.

Desta forma, basta apenas criarmos grupos para todos os planetas, utilizando transformações de *scale* para definirmos os seus tamanhos, operações de *translate* para movê-los para as suas corretas posições e operações de *rotate* para conseguirmos alterar as posições dos anéis. No caso da Lua da Terra, esta herda as transformações de *translate* e *scale* do planeta Terra.

```
<!-- Terra -->
<group>
  <transform>
    <translate x="9" y="0" z="0"/>
    <scale x="0.56" y="0.56" z="0.56" />
  </transform>
  <models>
    <model file="sphere_1_10_10.3d" /> <!-- generator sphere 1 10 10 sphere_1_10_10.3d -->
  </models>
<!-- Lua -->
<group>
  <transform>
    <translate x="0" y="0" z="-2"/>
    <scale x="0.153" y="0.153" z="0.153" />
  </transform>
  <models>
    <model file="sphere_1_10_10.3d" /> <!-- generator sphere 1 10 10 sphere_1_10_10.3d -->
  </models>
</group>
</group>
```

Figura 3.3: Posicionamento da Terra e da Lua

É importante realçar que como os anéis apenas são visíveis na face superior, devido à ordem em como os triângulos foram definidos, posicionámos outro anel na mesma posição mas aplicámos uma rotação de 180°, de modo a inverter a orientação dos triângulos e garantir que a face inferior seja renderizada corretamente, com a ordem dos triângulos invertida.

```
<!-- Saturno -->
<group>
  <transform>
    <translate x="22" y="0" z="0"/>
    <scale x="1.17" y="1.17" z="1.17" />
  </transform>
  <models>
    <model file="sphere_1_10_10.3d" /> <!-- generator sphere 1 10 10 sphere_1_10_10.3d -->
    <model file="ring_15_1p5_0p2.3d" /> <!-- generator ring 15 1.5 0.2 ring_15_1p5_0p2.3d -->
  </models>
  <group>
    <transform>
      <rotate angle="180" x="1" y="0" z="0"/>
    </transform>
    <models>
      <model file="ring_15_1p5_0p2.3d" /> <!-- generator ring 15 1.5 0.2 ring_15_1p5_0p2.3d -->
    </models>
  </group>
</group>
```

Figura 3.4: Posicionamento de Saturno e o seu anel

3.3 Execução da *demo scene*

A *demo scene* está localizada no diretório tests e, para ser executada basta correr o seguinte comando para gerar os objetos dos planetas:

- `./generator sphere 1 10 10 sphere_1_10_10.3d`
- `./generator ring 15 1.5 0.2 ring_15_1p5_0p2.3d`
- `./generator ring 15 1.3 0.5 ring_15_1p3_0p5.3d`

Por fim, basta correr o seguinte comando para visualizar o sistema solar:

- `./engine solar_system.xml`

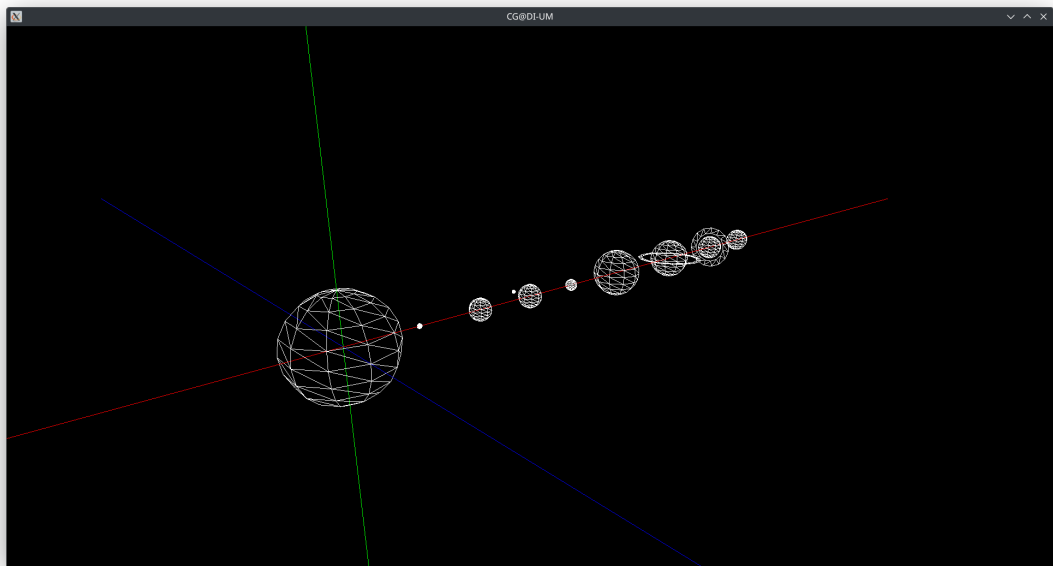


Figura 3.5: Sistema solar

4 Conclusão

Nesta segunda fase do projeto, conseguimos implementar com sucesso as transformações geométricas de **rotation**, **translate** e **scale**, permitindo um maior controlo sobre a organização e posicionamento dos modelos. Para tal, realizámos alterações na estrutura do **parser**, adicionando uma nova estrutura **Transform** dentro dos grupos do XML. Esta modificação garantiu que cada grupo pudesse conter no máximo três transformações (uma de cada tipo) e que os grupos filhos pudessem herdar as transformações dos seus grupos pai.

Além da implementação das transformações, conseguimos identificar e corrigir erros na definição dos triângulos da **box**, que passaram despercebidos na primeira fase do projeto. Estes erros foram evidenciados pela criação das **demo scenes**, demonstrando a importância da visualização e da validação contínua durante o desenvolvimento.

A geração do sistema solar representou o maior desafio desta fase, exigindo a criação de um modelo adequado para os anéis de Saturno e Urano. Para a construção dos anéis, aplicámos o conceito de duas circunferências com **inner_radius** e **out_radius**, e utilizámos a decomposição destas em **slices** para a correta modelação com triângulos. Com base neste modelo, conseguimos definir as coordenadas necessárias para cada ponto do anel através de funções trigonométricas.

De forma geral, esta fase consolidou o nosso conhecimento sobre manipulação de transformações geométricas, permitindo-nos criar um sistema solar funcional e visualmente coerente.